Prueba Técnica: Ingeniero de Visión por Computadora

Bienvenido/a al desafío técnico de Deliryum.AI.

En Deliryum.AI **NO** buscamos solo modelos que funcionen en un entorno controlado (Jupyter Notebook); construimos sistemas que operan en los entornos más exigentes de la industria pesada y la minería.

El objetivo de esta prueba es evaluar tu capacidad para diseñar, investigar, comparar y desplegar un **Pipeline de Visión Artificial de Alto Rendimiento para Análisis Granulométrico**. Buscamos rigor metodológico, calidad de ingeniería y capacidad para tomar decisiones basadas en datos empíricos.

1. Contexto de La Prueba técnica: "Lithos Analytics" 🪨

En el procesamiento de minerales, conocer la distribución del tamaño de las rocas (granulometría) que ingresan en vehículos es vital para la eficiencia energética y operativa. Actualmente, tienes acceso a imágenes de rocas depositadas en tolvas .

Tu misión es construir el núcleo **"Lithos Analytics"**: capaz de procesar estas imágenes, segmentar el material, calcular métricas estandarizadas de la industria y empaquetar.

2. Requisitos Tecnológicos (Stack) 💻

- **Lenguaje:** Python 3.9+. o framework a elección.
- **Inferencia/Optimización (Clave para Seniority):** Se valorará el uso de TensorRT, ONNX Runtime o PyTorch, TensorFlow o estrategias de optimización de inferencia para hardware de borde.
- **Despliegue:** Docker (Requisito excluyente). El sistema debe ser "Plug & Play".
- **Calidad de Código:** Tipado estático (Typing), Docstrings, y principios SOLID.

3. El Desafío Técnico 🛠️

Deberás construir un pipeline que reciba una carpeta con imágenes de rocas y genere métricas automatizadas. El sistema se divide en tres módulos críticos:

Módulo 1: Benchmarking de Métodos (Investigación Aplicada)

No existe una única forma correcta de segmentar rocas. Queremos ver cómo abordas el problema.

- **Tarea:** Implementa y evalúa al menos **tres (3) métodos distintos** para la segmentación y cálculo granulométrico. En es especial ML y DL (por las condiciones del Cargo) evitar el

uso de procesos algoritmicos baciscos, salvo que se apliquen para mejorar los resultados y no como un método BASE.

- **Métricas:** Selecciona a tu criterio las métricas de evaluación visual y paramétrica (ej. IoU, mAP, error en D50).
- **Output:** Un informe crítico (reporte_benchmark.md o PDF) justificando la selección de hiperparámetros, métricas y presentando un análisis del *trade-off* entre precisión y costo computacional de los 3 métodos.

Módulo 2: Proceso de Inferencia

El sistema debe procesar las imágenes de una carpeta de *Input* y generar de forma automatizada los siguientes archivos en una carpeta de *Output*:

1. **Imagen Segmentada:** La imagen original con las máscaras/contornos de las rocas superpuestos.
2. **Gráfico de Curva Granulométrica (Acumulada):** Gráfico (Eje X: tamaño en píxeles, Eje Y: % pasante acumulado) indicando los puntos de corte estandarizados **D20, D50 y D80**.
3. **Gráfico de Distribución:** Histograma o curva de densidad del tamaño de las rocas.
4. **Grafico de deformidad de Rocas:** Generar un scores y calificar la excentricidad y deformidad de las rocas.
5. **Datos Tabulares:** Un archivo .csv por cada gráfico generado con los datos crudos.
6. **Metadatos Operativos:** Un archivo .json que incluya tiempos de inferencia por imagen y el método utilizado....

*(Nota: **No** se requiere interfaz gráfica o Dashboard. Evaluamos el backend y el procesamiento, si la haces es más para tu facilidad).*

Módulo 3: MLOps y Despliegue Estricto

Tu solución será evaluada ejecutando tu contenedor contra un **Set de Testing oculto**.

- **Tarea:** El contenedor debe compilar a la primera y exponer un punto de entrada (CLI) que permita configurar dinámicamente rutas y seleccionar el método a testear (de la lista de tu bechmark).
- **Ejemplo de ejecución esperada:**
  docker run -v /ruta/local/input:/app/input -v /ruta/local/output:/app/output tu_imagen_docker python main.py --input /app/input --output /app/output --method metodo_2

4. Entregables y Reglas 📦

- **Repositorio de Código (Git):** Enlace al repositorio con tu solución.
- **Dockerfile:** Capaz de autogestionar todas las dependencias del sistema.
- **Reporte de Benchmark:** El análisis comparativo del Módulo 1.
- **Uso de IA / Datos Externos:** Totalmente **permitido**. Puedes usar ChatGPT, Copilot, u otros para codificar. Puedes **usar datasets públicos** para entrenar o hacer *fine-tuning (no olvides citarlos)*. Evaluamos la arquitectura, integración y el criterio de selección, no la memorización de sintaxis.

5. Criterios de Evaluación

No buscamos un script frágil

- **Arquitectura de Software y Calidad de Código (25%):** ¿Es modular? ¿Se aplican patrones de diseño? Tipado, manejo de excepciones y limpieza.
- **Rigor Metodológico y Benchmark (35%):** Profundidad del análisis del Módulo 1. Justificación matemática/algorítmica para el cálculo preciso del D20, D50 y D80.
- **Precisión y Performance (37%):** Calidad visual de la segmentación. Tiempos de inferencia optimizados (FPS teóricos). Correcto balance entre peso computacional y exactitud.
- **Robustez MLOps y Contenerización (13%):** Ejecución "Out-of-the-box". Si el Docker falla al ejecutar el comando principal por problemas de dependencias o rutas hardcodeadas, habrá una penalización severa.

¿Dudas? Si algo no está especificado, asume el mejor criterio técnico y documéntalo en tu README. En operaciones críticas, tu capacidad para tomar decisiones con información incompleta es fundamental.

Si las dudas persisten, contactarme en el mail: benjamin.huerfano@deliryum.ai

¡Éxito en el desafío! Estamos ansiosos por revisar tu código.